

SKILLRC: Is It the Skill or the Stack?

Auditing Experience Memory in Agent Harnesses

Yiwen Lu^{*1}

¹University of Notre Dame

Manuscript, October 2026.

<https://eveneureka.github.io/SkillRC/>

Abstract

Agent harnesses increasingly learn from experience: past executions are distilled into insights, skills, or playbooks that are injected back into the context of a frozen model. Such memories are usually credited by comparing an agent with the memory to one without it. That comparison answers only the first of three questions: whether the system improves, whether the gain carries over to a different executor, and how much of it comes from what the memory says rather than from adding context. We introduce SKILLRC, a reality check (RC) for agent skills that asks all three of a fixed agent. It treats the memory as a controlled intervention, estimates its gain on two executor stacks, and compares the true memory with a placebo that keeps the memory’s rules, order, and length but scrambles its words. On ALFWorld, a compact skill memory clearly improves GPT-4o-mini on every demonstration pair, with the smallest payload we test. The same memory does not help Qwen3-8B, and the placebo already separates the two executors by as much as the true memory does, while coherent content adds a modest increment to both that our intervals cannot distinguish from zero. The gain is therefore real and specific to the executor stack, and its source becomes a question the protocol can test rather than one a with-versus-without comparison can settle. Task-level effects vary widely, larger memories are not better, and the same kind of memory lowers reward on WebShop. Experience-driven harnesses should report cross-executor effects and content controls alongside their headline gains.

1 INTRODUCTION

The performance of an LLM agent depends not only on the underlying language model but on the *agent harness* that surrounds it: the control layer that constructs context, exposes tools and retrieval, orchestrates calls, retains memory, and processes outputs (Huang et al., 2026; Lee et al., 2026). A growing line of work makes this layer *learn from experience*. ExpeL distills natural-language insights from past trajectories and recalls them at inference (Zhao et al., 2023); Reflexion turns feedback into verbal memory (Shinn et al., 2023); SkillOpt and SkillOS edit or curate skill libraries around a frozen executor (Yang et al., 2026; Ouyang et al., 2026); and MEMOHARNESS accumulates an experience bank that it uses to search over and adapt the harness itself (Huang et al., 2026). These systems share one mechanism: execution experience is converted into text that re-enters the model’s context.

Despite this progress, the evidence that an experience memory helps is usually a with-versus-without comparison on a single base model, and the memory is rarely separated from the other changes it brings to the prompt. Skill benchmarks find small, heterogeneous, or negative marginal value at substantial token cost (Li et al., 2026; Han et al., 2026; Liu et al., 2026), but they cannot say *why* a skill

helps one configuration and hurts another. What is missing is a measurement layer for the experience component itself: one that asks whether an experience edit to the harness transfers across executors, and whether its gain comes from what the experience says.

This setting creates three challenges. First, **the executor is a stack**, not a model: weights, tokenizer, chat template, serving runtime, and decoding change together, and the same memory may help one stack while disrupting another. Second, **a with-versus-without contrast confounds content with context**: language models are sensitive to spurious prompt features (Sclar et al., 2023) and to irrelevant context (Shi et al., 2023), and when a memory pool fits inside the context budget, every retriever injects the same items and differs only in their order. Third, **aggregate gains hide transfer structure**: a positive mean can combine rescues on some tasks with harm on others.

We address this gap with SKILLRC, a *reality check* (RC) for agent skills: a controlled experience-memory harness and evaluation protocol. SKILLRC makes three design choices. **(1) Experience memory as a controlled harness intervention.** We describe the harness by the factors a memory comparison can change, namely the executor stack and the memory’s content, representation, presentation, and footprint, vary one factor at a time on identical tasks, and audit the items, order, and tokens that actually reach the model (§2.3). **(2) Interaction-first transfer estimation.** The focal quantity is the *transport gap*, the difference between two executor stacks’ paired memory gains on identical tasks, prompts, and memory, estimated with task-clustered uncertainty (§2.5). **(3) A pre-registered order-placebo gate.** Before outcomes are inspected, we freeze a placebo that preserves item membership, retrieval order, and token footprint while removing every original adjacent word pair, together with the rule for attributing a gain to content (§2.6). Figure 1 summarizes the protocol.

We evaluate SKILLRC on ALFWorld (Shridhar et al., 2021) with two executor stacks and on WebShop (Yao et al., 2022). The experience memory improves the harness for GPT-4o-mini, raising success from 0.587 to 0.714 on every demonstration pair, and the most compact memory, 33 rules in 645 tokens, is also the most effective one. The improvement, however, is specific to the stack: the same memory leaves Qwen3-8B unchanged, and a placebo that keeps the memory’s rules, order, and length but scrambles its words already separates the two stacks by as much as the true memory does, while coherent content adds a modest increment to both whose intervals include zero. We read these results as a progression rather than as a sequence of failures. The first establishes that the memory helps; the second that the gain is an interaction between memory and executor stack, which a single-model comparison cannot see; and the third that the share of this interaction owed to the experience content is the quantity still to be pinned down, which the protocol makes testable.

Contributions. The main contributions of this work are as follows:

- A *controlled experience-memory harness* that describes a memory comparison by explicit factors (executor, content, representation, presentation, footprint), varies one at a time, and audits the realized payload through a SHA-256 payload passport (§2.3 and §2.4).
- An *interaction-first evaluation protocol* that estimates executor transport gaps with task-clustered intervals and gates content claims behind a payload-matched order placebo frozen before outcomes (§2.5 and §2.6).
- Empirical evidence that a compact skill memory improves one executor but not another, that a content-free placebo already opens a gap of similar size, so that the executor interaction cannot be attributed to differential use of content on this evidence, that transfer varies widely across tasks, and that the same kind of memory can hurt in a second environment (§3).

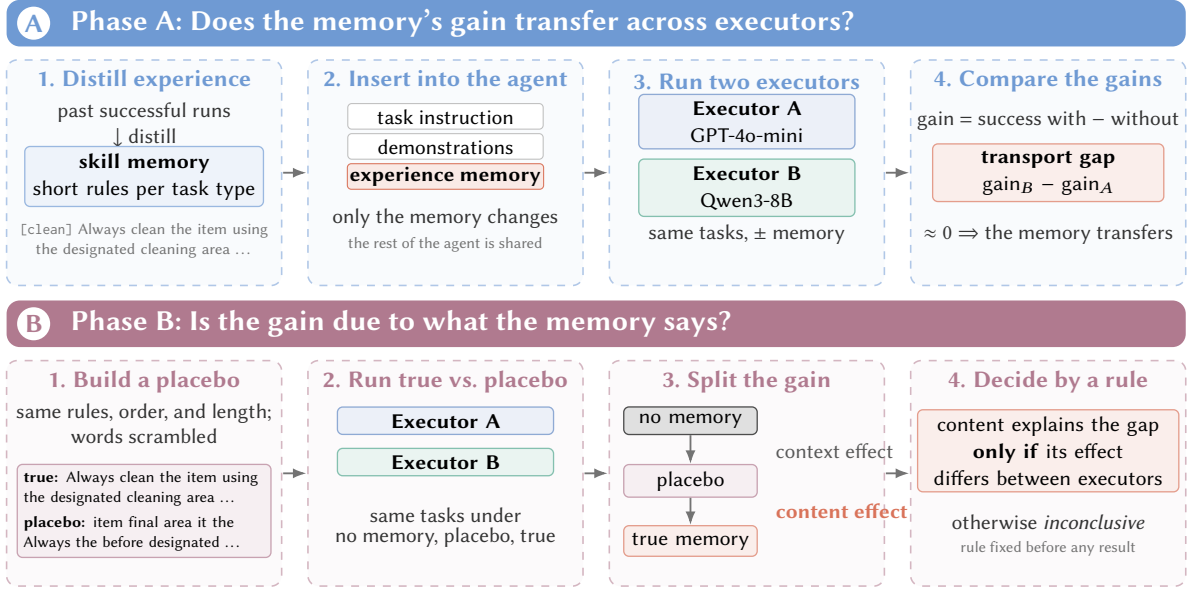


Figure 1. Overview of SKILLRC. **Phase A** asks whether a memory’s gain transfers: past successful runs are distilled into a skill memory, inserted into an agent whose other parts stay the same, and run with and without the memory on two executors; the difference between their gains is the *transport gap*. **Phase B** asks whether the gain comes from what the memory says: a placebo keeps the same rules, order, and length but scrambles the words, which splits the gain into a *context effect* (no memory → placebo) and a *content effect* (placebo → true memory), and a rule fixed in advance decides whether content explains the gap.

2 METHOD

Overview. We study *experience memory* as one component of an agent harness: a pool of text distilled from past executions that a frozen executor reads at inference. SKILLRC treats the memory as a controlled edit to a harness whose other dimensions are fixed, executes the edited and unedited harness on identical tasks under several executor stacks, and estimates how the paired gain changes across stacks. It follows a **content-attribution-first** principle: a gain is credited to the experience content only if it survives a payload-matched placebo whose decision rule is fixed in advance. Figure 1 shows the two phases of the protocol.

2.1 Motivation

Experience memories are usually reported as properties of an artifact: a pool is built once, injected, and credited with the gain it produces. This treats the artifact as portable across executors and as acting through its content. Both assumptions are testable. Portability is an interaction between the memory and the executor stack, and content use is a contrast between the memory and a control that carries the same prompt-level intervention without its content. SKILLRC makes both contrasts explicit and holds everything else in the harness fixed, so that a reported gain names exactly which harness dimensions produced it.

2.2 Problem Formulation

Definition 2.1 (Evaluation Cases and Executor Stacks). Let $\mathcal{T} = \{t_1, \dots, t_n\}$ be evaluation tasks, disjoint from the executions used to build any memory. An executor stack $E \in \mathcal{E}$ bundles model weights, tokenizer, chat template, serving runtime, and decoding parameters. In this paper $\mathcal{E} = \{G, Q\}$, with G denoting

GPT-4o-mini and Q denoting Qwen3-8B.

Definition 2.2 (Harness Configuration). A SKILLRC configuration is

$$H = (E, M, d), \quad M = (c, r, p, B), \quad (1)$$

an executor stack E , a memory condition M , and a demonstration pair d . The memory condition fixes the content $c \in \{\text{none}, \text{true}, \text{placebo}\}$, the representation r , the presentation p , and the token budget B (Section 2.3). The agent loop, decoding, and action parsing are shared by every configuration and are not factors.

Definition 2.3 (Experience Memory and Realized Payload). A memory condition M is inserted as one block after the task scaffold and changes nothing else in the prompt. We write $M = 0$ for no memory and $M = P$ for the order placebo, which has placebo content and the same representation, presentation, and budget as the true memory. For task t , the realized payload is

$$\pi_t(M) = (I_t, \sigma_t, \ell_t), \quad (2)$$

the injected item set, its order, and its token count. Two conditions are payload-matched if I_t and σ_t coincide for every t and ℓ_t is equal under every tokenizer in the study.

Executing task t under stack E , memory M , and demonstration pair d produces an environment-native outcome and a trajectory with lightweight runtime diagnostics,

$$\tau_{E,M,t,d} = (Y_{E,M,t,d}, \mathcal{A}_{E,M,t,d}, \kappa_{E,M,t,d}), \quad \kappa = (n^{\text{step}}, n^{\text{prompt}}, \ell_t), \quad (3)$$

where $Y \in \{0, 1\}$ is success on ALFWorld and $Y \in [0, 1]$ is reward on WebShop, $\mathcal{A}$ is the action-observation trace, and κ records environment steps, cumulative prompt tokens, and realized memory tokens. Cost enters the analysis only as a reported quantity, never as a selection criterion.

2.3 Harness Factors

Design rationale. A memory comparison changes more than “the memory”. It changes which items reach the model, in which order, and how much context they occupy, and its effect is read by a particular executor stack. We therefore describe the harness by the factors a memory comparison can change and vary one factor at a time, keeping the rest of the agent identical by construction.

Two consequences follow from these factors (Table 1). First, presentation claims require a membership audit: the focal pool holds 33 items and 645 tokens, so under the 2,048-token budget both BM25 and dense ranking inject the complete pool, and their contrast changes only the order. Second, nominal budgets do not equalize footprint: compact insights realize 645 tokens, raw trajectories about 1,779, and Markdown procedures 1,970–1,989 under the same budget, so a representation comparison also changes how much context the memory occupies.

2.4 Realized-Payload Audit

Every episode logs the injected text, its token count under the executor’s tokenizer, and the retrieval order. A *payload passport* records SHA-256 hashes of the pool and payload together with the retriever, budget, and item cap; in strict mode a run is rejected when the executor or payload hash does not match the card. The audit turns “we used BM25 over the skill pool” into a checkable statement about which items reached the model, in which order, and at what cost.

Table 1. Factors of the SkillRC harness. Each comparison varies one factor on identical tasks. The agent loop (ReAct with admissible actions and a fixed step limit), decoding (temperature 0, 256 completion tokens per turn), and action parsing are shared by every configuration and are not factors.

Factor	Role	Definition and levels in this study
E Executor stack	varied (transfer)	The model stack that reads the memory: weights, tokenizer, chat template, serving runtime. GPT-4o-mini (hosted API) Qwen3-8B (vLLM, thinking off)
c Memory content	varied (attribution)	Whether the injected items carry coherent experience. none true insights order placebo
r Memory representation	varied (exploratory)	What an item is: a distilled rule, a raw trajectory, or a procedure. 33 compact insights raw trajectories Markdown procedures
p Presentation	varied (exploratory)	How items are ranked for the task; when the pool fits in B, this sets order, not membership. BM25 dense reference-ranked (placebo)
B Footprint	measured and matched	The nominal budget and the tokens actually injected. B = 2048; realized 645 (insights), 1779-1780 (trajectories), 1970-1989 (Markdown)
d Demonstration pair	averaged (nuisance)	Which two of three ReAct demonstrations the prompt shows. pairs 01 02 12, weighted equally

2.5 Interaction-First Transfer Estimation

The scaffold selects two of three demonstrations per task type, so there are three unique demonstration pairs $d \in \mathcal{D} = \{01, 02, 12\}$. We average repeated executions within a pair and weight pairs equally,

$$\bar{Y}_{E,M,t} = \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} Y_{E,M,t,d}. \quad (4)$$

The paired memory gain of stack E and the *transport gap* between stacks are

$$U_E(M) = \mathbb{E}_t [\bar{Y}_{E,M,t} - \bar{Y}_{E,0,t}], \quad \Delta_{E \times M} = U_Q(M) - U_G(M). \quad (5)$$

$\Delta_{E \times M}$ is the focal portability estimand: a memory transfers when the gap is near zero, regardless of whether each stack’s gain is separately significant. Intervals are percentile intervals from 10,000 resamples of task clusters, so repeated prompt conditions are never treated as independent seeds.

2.6 Order-Placebo Gate

The placebo shuffles the body words of each item until no original adjacent body bigram remains, keeping task tags, item indices, and word counts. Shuffling lengthens the payload from 645 to 673 BPE tokens, so a deterministic greedy pass replaces 28 of 472 body words with a nonce, restoring exactly 645 tokens while retaining 94.1% of body words. To hold presentation fixed, the placebo is not re-indexed: BM25 ranks the *original* pool as a read-only reference, and the placebo items are injected at the same indices. Comparing the true memory with the placebo defines the *coherent-content increment*

$$S_E = \mathbb{E}_t [\bar{Y}_{E,M,t} - \bar{Y}_{E,P,t}], \quad \Delta_{E \times S} = S_Q - S_G. \quad (6)$$

Frozen decision rule. Executor-dependent use of experience content is claimed only if the interval for S_G excludes zero positively *and* the interval for $\Delta_{E \times S}$ excludes zero negatively. True and placebo are treated as equivalent for GPT-4o-mini only if the interval for S_G lies inside $[-0.05, +0.05]$. Otherwise the mechanism is *inconclusive*.

The task sample (ten tasks per type, RNG seed 20260713, chosen from task IDs and types only), the estimands, the stratified bootstrap, and the rule were frozen before any placebo outcome was read. Two earlier payloads were stopped before outcome inspection: one left only 31 items complete after truncation, and the other replaced 74.4% of words with a repeated nonce. None of their episodes enter the analysis.

3 EXPERIMENTS

We organize the evaluation around five research questions that build on one another. RQ1 asks whether experience memory improves the harness. RQ2 examines whether the improvement transfers across executor stacks. RQ3 tests how much of the stack-specific gain the content of the experience explains. RQ4 asks where transfer succeeds or fails across tasks. RQ5 measures whether more experience context buys more utility and whether the pattern appears in a second environment.

3.1 Setup

We evaluate SkillRC on the 134-task unseen split of ALFWorld (Shridhar et al., 2021), covering six task types, with a ReAct scaffold (Yao et al., 2023) that sees admissible actions. The executors are the hosted GPT-4o-mini API and locally served Qwen3-8B, both at temperature zero with Qwen thinking disabled. The focal memory is a pool of 33 compact ExpeL-style insights distilled from successful trajectories on the seen validation split; exploratory conditions use raw successful trajectories and SkillOS-style Markdown procedures built from the same source. For RQ5 we use WebShop-small with 1,000 products (Yao et al., 2022). All effects are paired on tasks. We report the no-memory harness as the baseline for each stack and, for content attribution, the payload-matched placebo. Appendix E gives benchmark, model, and protocol details.

3.2 Results

RQ1 Does experience memory improve the harness?

Figure 2a compares the no-memory harness with the experience-memory harness under both executor stacks. For GPT-4o-mini, the memory raises success from 0.587 to 0.714, a paired gain of +0.127 ([+0.072, +0.184]). The gain is present on every demonstration pair (+0.112, +0.097, and +0.172 on pairs 01, 02, and 12), so it is not produced by one favorable prompt. It is also obtained with the smallest payload we test: the compact insights realize 645 memory tokens, whereas raw trajectories and Markdown procedures realize 1,779–1,989 tokens under the same budget and none of them exceeds this gain (RQ5).

The improvement is therefore real within the stack where the memory was built: the same frozen executor, the same scaffold, and the same tasks become more successful when 33 distilled rules are placed in context. This is the effect that experience-driven harnesses are designed to produce, and it is the reference point for the questions that follow.

RQ2 Does the improvement transfer across executor stacks?

The gain is specific to the stack it was built with. Under Qwen3-8B, the same memory moves success

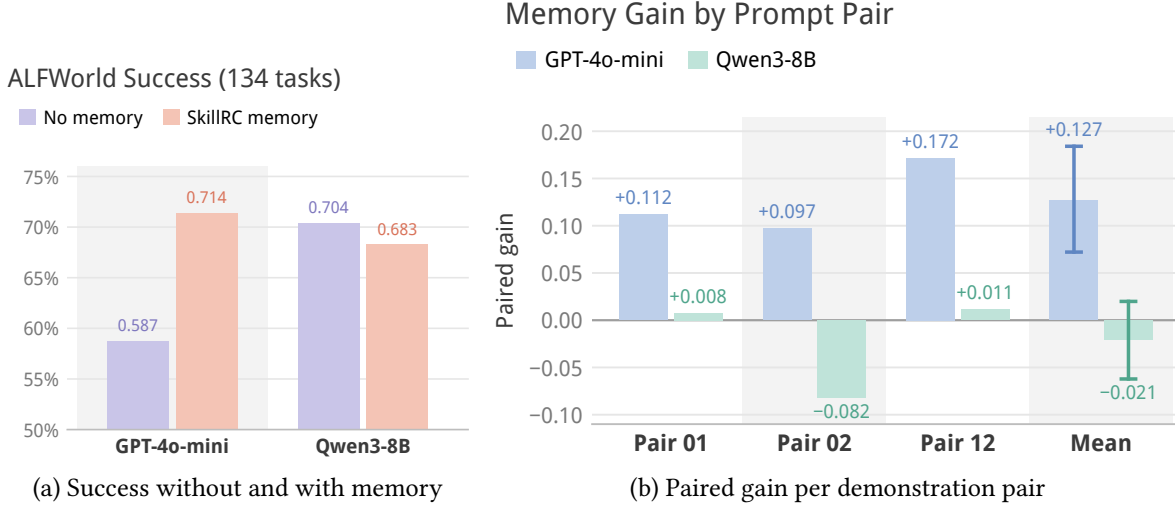


Figure 2. ALFWorld results for the 645-token experience memory. (a) Equal-pair success: the memory raises GPT-4o-mini from 0.587 to 0.714 but not Qwen3-8B. (b) Paired gain on each demonstration pair and the equal-pair mean with its 95% task-bootstrap interval. GPT-4o-mini gains on every pair; the transport gap $\Delta_{E \times M} = -0.148$ excludes zero.

from 0.704 to 0.683, a change of -0.021 ($[-0.062, +0.020]$), and the pairwise effects are $+0.008$, -0.082 , and $+0.011$ (Figure 2b). The transport gap is $\Delta_{E \times M} = -0.148$ ($[-0.218, -0.080]$). The focal contrast was designated after a seed-axis audit showed that two nominal seeds selected the same demonstration pair (Appendix M); using only the two originally observed pairs gives a gap of -0.170 ($[-0.252, -0.088]$).

Observation. The improvement does not transfer, and the gap is not explained by one prompt pair. The second stack starts from higher no-memory success (70.4% against 58.7%), leaving less headroom, and RQ3 shows that it also responds to the added block differently. The interval identifies moderation of the whole payload by the stack; it does not identify which stack component, whether weights, tokenizer, template, or runtime, is responsible.

RQ3 Is the improvement driven by experience content?

The placebo already separates the two stacks, and coherent content adds a modest increment to both. On the 60-task gate the true memory reproduces the original direction: GPT-4o-mini rises from 0.544 to 0.681 ($+0.136$, $[+0.075, +0.200]$) and Qwen3-8B falls from 0.728 to 0.683 (-0.044 , $[-0.100, +0.006]$). The placebo, which carries the same items in the same order at the same footprint but no coherent text, already moves the stacks apart: $+0.089$ for GPT-4o-mini ($[+0.025, +0.153]$) and -0.108 for Qwen3-8B ($[-0.164, -0.053]$; Figure 3). Coherent content then adds $S_G = +0.047$ for GPT-4o-mini ($[-0.019, +0.117]$; 21/16/23 task wins, losses, and ties) and $S_Q = +0.064$ for Qwen3-8B ($[0.000, +0.128]$; 18/11/31). The gap in this increment is $\Delta_{E \times S} = +0.017$ ($[-0.075, +0.108]$).

Observation. Under the frozen rule the content mechanism is *inconclusive*: coherent text adds a positive increment for both stacks, but both intervals include zero, and the executor reversal of RQ2 cannot be attributed to differential use of experience content on this evidence. In point estimates, the placebo alone opens a stack gap of -0.197 , as large as the true memory’s -0.181

Order-Placebo Gate (60 frozen tasks)

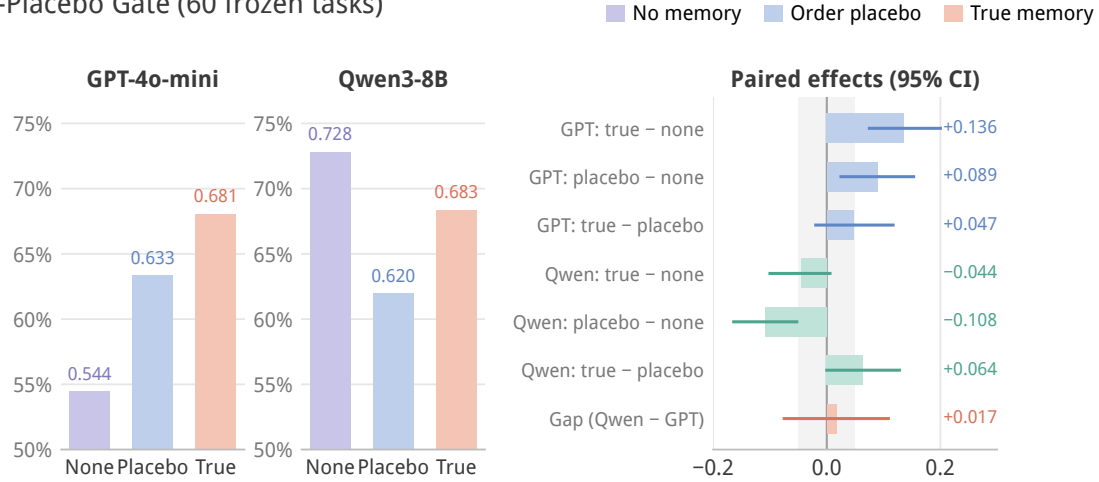


Figure 3. Pre-registered order-placebo gate on 60 frozen tasks. Left: equal-pair success under no memory, the order placebo, and the true memory; placebo success equals no-memory success plus the reported placebo effect. Right: paired effects with 95% stratified task-bootstrap intervals; the grey band is the frozen equivalence region $[-0.05, +0.05]$.

on the same tasks, so the interaction between memory and executor is established while the share of the gain owed to content is not. A scrambled block is also not a neutral control: it may distract differently from coherent text, and it keeps the memory’s vocabulary. The comparison therefore turns the source of the gain into a testable question rather than settling it; the next control is a coherent, equal-length memory that is irrelevant to the task.

RQ4 Where does transfer succeed or fail across tasks?

Transfer is heterogeneous (Figure 4). Only 9 tasks improve under both stacks and 6 decline under both. Forty-five improve under GPT-4o-mini but are neutral (31) or harmed (14) under Qwen3-8B, and 17 show the converse. Thirteen of the 14 tasks that help GPT-4o-mini and hurt Qwen3-8B have Qwen3-8B no-memory success of at least two thirds, against a mean of 0.407 among tasks that help both, which is compatible with a ceiling-plus-interference account but does not establish it. Task types differ as well: GPT-4o-mini gains range from +0.255 on putt two and +0.246 on heat to -0.111 on cool, and the types with the weakest GPT-4o-mini baseline gain the most. Cross-fitted difficulty, which classifies each observation using only the other two no-memory observations of the same task, shows GPT-4o-mini gains declining from +0.261 on hard tasks to +0.157 on mixed and +0.048 on easy ones, while Qwen3-8B shows no monotone pattern (Appendix K; Appendix L walks through one case per task type).

RQ5 Is more experience context better, and does the pattern hold elsewhere?

More context does not buy more utility (Table 2). The strongest GPT-4o-mini condition injects the fewest tokens, while trajectory and procedural memories, roughly three times larger under the same nominal budget, never exceed the compact BM25 gain, and Markdown procedures harm Qwen3-8B under both presentations (-0.077 and -0.062 , both intervals below zero). Because the 33 insights always fit, BM25 versus dense ranking changes only their order, yet order alone moves the GPT-4o-mini gain from +0.147 to +0.070; the executor-by-presentation interaction is -0.092 ($[-0.162, -0.020]$). In a balanced 12-task matched-footprint pilot, no memory and all three compressed memories each solve

Task-Level Transfer (134 ALFWorld tasks)

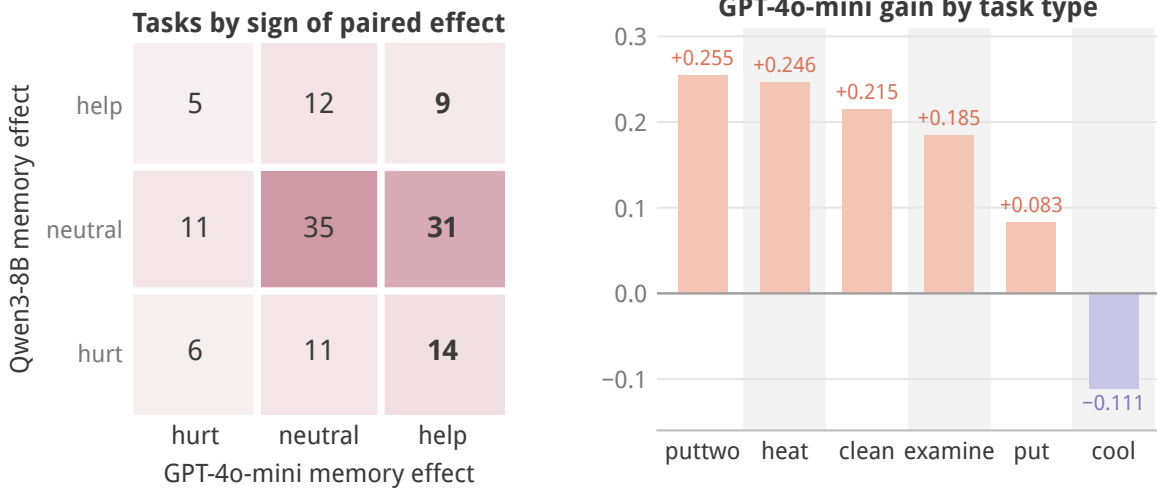


Figure 4. Task-level transfer. Left: the 134 ALFWorld tasks by the sign of their equal-pair memory effect under each stack. Right: GPT-4o-mini gain by task type in the nominal-seed matrix.

Table 2. Realized memory tokens and paired success gains for each memory representation and presentation in the exploratory matrix (nominal seeds 0–2; intervals in Appendix J). Raw trajectories were not run under Qwen3-8B. The highlighted row is the focal SKILLRC memory.

Representation	Presentation	Memory tokens	GPT-4o-mini Δ	Qwen3-8B Δ	Gap
Compact insights	BM25	645	+0.147	-0.020	-0.167
Compact insights	dense	645	+0.070	-0.005	-0.075
Raw trajectories	BM25	1,780	+0.072	–	–
Raw trajectories	dense	1,779	+0.105	–	–
Markdown procedures	BM25	1,989	+0.047	-0.077	-0.124
Markdown procedures	dense	1,970	+0.097	-0.062	-0.159

5/12 tasks, but different ones, which points to task reallocation rather than equal-footprint superiority. The memory is also cheap for the stack it helps: GPT-4o-mini episodes become shorter (18.1 against 20.3 steps), so the 645-token block raises total prompt tokens by 16.1% and API cost by 9.6%, most of it cached (Appendix F).

In WebShop, a 20-task gate first confirmed that Qwen3-8B executes validly (100% valid action syntax, search on 20/20 tasks, 13 purchase attempts, mean reward 0.288). We distilled ten ExpeL-style rules from the six positive-reward trajectories of train sessions 500–519 and compared no memory with this pool on disjoint sessions 0–99. Reward falls from 0.194 to 0.105 (paired -0.090 , $[-0.165, -0.016]$), with 8 improvements, 20 declines, and 72 ties, and exact success moves from 0.09 to 0.07 (-0.020 , $[-0.090, +0.050]$; Figure 5).

Observation. Experience memory can reduce performance outright. The WebShop pool was distilled from only six positive trajectories and was read by the stack that also showed no ALFWorld gain; it has twice as many task losses as wins. This check uses one executor and no placebo, so it is a bounded negative-transfer replication rather than an estimate of executor

WebShop (Qwen3-8B, 100 train-disjoint sessions)

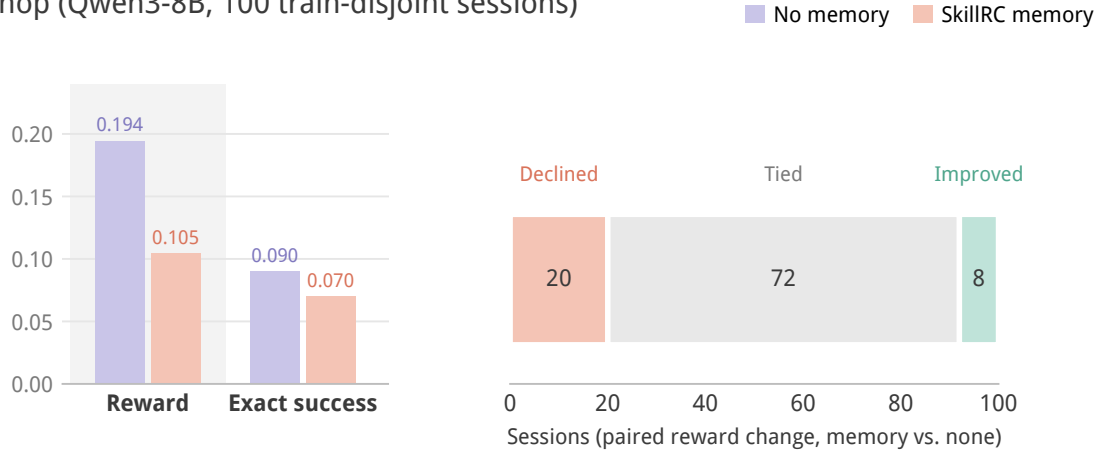


Figure 5. Train-disjoint WebShop comparison with local Qwen3-8B. Left: mean reward and exact purchase success without and with a ten-rule experience memory. Right: sign of the paired reward change over 100 sessions.

moderation in WebShop.

4 CONCLUSION

We presented SKILLRC, a reality check for experience memory in agent harnesses. It varies only the executor stack and the memory’s content, representation, presentation, and footprint, audits what reaches the model, and evaluates the memory through paired transport gaps and a pre-registered, payload-matched placebo. The three questions it asks build on one another. On ALFWorld a compact skill memory raises GPT-4o-mini success by 12.7 points, on every demonstration pair and with the smallest payload we test, so the memory does improve the system it was built with. The same memory does not help Qwen3-8B, so the gain is an interaction between memory and executor stack rather than a property of the memory alone. And a placebo that keeps the items, order, and footprint already separates the two stacks by as much as the true memory does, while coherent content adds a modest increment to both whose intervals include zero, so the share of the gain owed to what the memory says is the quantity that remains to be measured. A with-versus-without comparison answers only the first question; the effect of an experience memory has to be understood together with the stack that reads it. Future work should add a coherent, equal-length memory that is irrelevant to the task, to separate semantic relevance from textual interference, manipulate stack components one at a time, extend the placebo to regimes where retrieval changes membership, and apply the same audit to harnesses that learn their memories online.

REFERENCES

- Julia Belikova, Rauf Parchiev, Evgeny Egorov, Grigorii Davydenko, Gleb Gusev, Andrey Savchenko, and Maksim Makarenko. Managing procedural memory in LLM agents: Control, adaptation, and evaluation, 2026. URL <https://arxiv.org/abs/2606.23127>.
- Yanping Chen, Weijie Shi, Wen Yang, and Jiajie Xu. Task decomposition-guided reranking for adaptive agent skill retrieval, 2026. URL <https://arxiv.org/abs/2607.06283>.

- Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution, 2023. URL <https://arxiv.org/abs/2309.16797>.
- Haowen Gao, Haoran Chen, Can Wang, Shasha Guo, Liang Pang, Zhaoyang Liu, Huawei Shen, and Xueqi Cheng. SelfEvoSkill: Paired execution audits for skill revision without external outcome supervision, 2026. URL <https://arxiv.org/abs/2606.14239>.
- Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. SWE-Skills-Bench: Do agent skills actually help in real-world software engineering?, 2026. URL <https://arxiv.org/abs/2603.15401>.
- Qisheng Hu, Quanyu Long, and Wenya Wang. When continual learning moves to memory: A study of experience reuse in LLM agents, 2026. URL <https://arxiv.org/abs/2604.27003>.
- Yue Huang, Wenjie Wang, Han Bao, Yuchen Ma, Xiaonan Luo, Yi Nian, Haomin Zhuang, Zheyuan Liu, Yue Zhao, and Xiangliang Zhang. MemoHarness: Agent harnesses that learn from experience, 2026. URL <https://arxiv.org/abs/2607.14159>.
- Ryangkyung Kang, Hongcheol Cho, and Youngeun Kim. SkillRet: A large-scale benchmark for skill retrieval in LLM agents, 2026. URL <https://arxiv.org/abs/2605.05726>.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines, 2023. URL <https://arxiv.org/abs/2310.03714>.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026. URL <https://arxiv.org/abs/2603.28052>.
- Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, et al. SkillsBench: Benchmarking how well agent skills work across diverse tasks, 2026. URL <https://arxiv.org/abs/2602.12670>.
- Yujian Liu, Jiabao Ji, Li An, Tommi Jaakkola, Yang Zhang, and Shiyu Chang. How well do agentic skills work in the wild: Benchmarking LLM skill usage in realistic settings, 2026. URL <https://arxiv.org/abs/2604.04323>.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, Maohao Shen, Vishy Tirumalashetty, George Lee, Jiawei Han, Tomas Pfister, and Chen-Yu Lee. SkillOS: Learning skill curation for self-evolving agents, 2026. URL <https://arxiv.org/abs/2605.06614>.
- Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting, 2023. URL <https://arxiv.org/abs/2310.11324>.

- Maksim Shaposhnikov, Nicolas Fortuin, Simon Stipcich, Maria I. Gorinova, Amy Heineike, and Rob Willoughby. A framework for evaluating agentic skills at scale, 2026. URL <https://arxiv.org/abs/2606.17819>.
- Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context, 2023. URL <https://arxiv.org/abs/2302.00093>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning, 2021. URL <https://arxiv.org/abs/2010.03768>.
- Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive strategy for self-evolving agent skills, 2026. URL <https://arxiv.org/abs/2605.23904>.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents, 2022. URL <https://arxiv.org/abs/2207.01206>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Dexu Yu, Youhua Li, Zhaoyang Guan, Xianhao Lin, Jining Luan, Zihao Rao, Xuanqi Lan, Yang Ran, Bo Lan, Nai-Xin Zhai, Hanwen Du, Junchen Fu, Wenhao Deng, Yongxin Ni, and Chunxiao Li. SkillAudit: From fixed-suite benchmarking to skill-centered assessment, 2026. URL <https://arxiv.org/abs/2606.22613>.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. Aflow: Automating agentic workflow generation, 2024. URL <https://arxiv.org/abs/2410.10762>.
- Xing Zhang, Guanghui Wang, Yanwei Cui, Wei Qiu, Ziyuan Li, Bing Zhu, and Peiyang He. Experience compression spectrum: Unifying memory, skills, and rules in LLM agents, 2026. URL <https://arxiv.org/abs/2604.15877>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners, 2023. URL <https://arxiv.org/abs/2308.10144>.

APPENDIX CONTENTS

A	Limitations	p. 13
B	Implementation details	p. 13
C	Prompt settings	p. 14
D	Experience memory contents	p. 17
E	Experimental details	p. 18
F	Per-task-type results and cost	p. 18
G	Models used	p. 19
H	Comparison scope	p. 19
I	Related work	p. 19
J	Representation matrix and executor interactions	p. 21
K	Task-level transfer diagnostic	p. 21
L	Case studies	p. 21
M	Seed-axis audit	p. 33
N	Order-placebo construction	p. 33

A LIMITATIONS

This study has several limitations. First, the executor contrast bundles model weights, tokenizers, chat templates, serving systems, and generation repeatability; it establishes stack-level moderation, not a model-weight mechanism. Second, the focal contrast was designated after a seed-axis audit, and the representation matrix, task-type, difficulty, and taxonomy analyses are exploratory and not adjusted for multiplicity. Third, the study covers two executor stacks, one focal 33-item pool, and ALFWorld; it is not a population estimate over models or experience artifacts, and the representation reproductions omit the optimizers and curators of ExpeL and SkillOS. Fourth, the order placebo isolates coherent local order imperfectly: shuffled text may distract differently from coherent text, and retained words keep bag-of-words cues; the 60-task gate is also lower powered than the 134-task focal contrast and targets an equal-task-type effect. Fifth, the WebShop check uses one local executor, one train-distilled pool, the small environment, and one execution per condition. Finally, the hosted GPT alias is mutable and temperature-zero hosted generations are not perfectly deterministic, and the public release contains the harness, configurations, pools, and analysis code but not per-episode logs, so the figures are drawn from the frozen summary statistics.

B IMPLEMENTATION DETAILS

This appendix describes how the abstract components of Section 2, namely the harness configuration, the memory payload, the audit, and the placebo, are realized in our reference implementation. None of these choices alters the estimands; they specify how the abstract objects are instantiated in the experiments of Section 3.

Harness representation. A configuration $H = (E, M, d)$ is materialized as one YAML file: the executor endpoint (E), the memory kind and pool path (c, r), the retriever and an optional index-aligned ranking reference (p), the token budget B and item cap K , and the demonstration file and seed

(d). The remaining keys, namely decoding parameters, prompt style, admissible-action injection, step limit, and environment, take the same values in every configuration of a comparison. Unknown keys are rejected at load time so that a configuration cannot silently omit a setting.

Initialization. Every comparison starts from the no-memory configuration ($c = \text{none}$), whose prompt contains only the task scaffold and two demonstrations. A memory condition adds one block after the scaffold and changes nothing else; the executor client, decoding, ReAct loop, and action parsing are shared code paths.

Memory construction. The ALFWorld pools are distilled with GPT-4o-mini from successful trajectories on the seen validation split, using at most six trajectories per task type: 33 natural-language insights (ExpeL-style), raw successful trajectories, and one Markdown procedure per successful trajectory (SkillOS-style). The WebShop pool holds ten rules distilled by local Qwen3-8B.

Presentation and budget fill. All pool memories share one fill path. Items are ranked, appended greedily until the next item would exceed B (or K items when no budget is set), and the concatenation is truncated to B tokens keeping its beginning. Only the ranking differs across presentations: whitespace-tokenized BM25, dense cosine similarity over text-embedding-3-small vectors, or lexical overlap. When a ranking reference is supplied, BM25 is fit on the reference pool and the payload items are returned at the reference indices, which is how the placebo inherits the true memory’s order exactly.

Payload accounting. Realized memory tokens are counted with the executor-appropriate tokenizer (tiktoken for GPT-4o-mini, the native tokenizer for the Qwen3-8B audit) and logged per episode, so footprint is reported as realized rather than nominal. The executor client also records prompt, completion, and cached tokens for every call and forwards `chat_template_kwargs` to disable Qwen3 thinking.

C PROMPT SETTINGS

This appendix reproduces every prompt used in the study, verbatim from the released code. All prompts are shared by every memory condition; a memory condition only appends the block headed `# Retrieved experience / skills`, whose items and order are set by the memory’s representation and presentation. Angle brackets mark slots that are filled at run time.

ALFWorld agent (ReAct, both executors). Each call sends two messages. The system message holds the instruction, the two demonstrations selected for the task type, and the memory block; it is constant within an episode, which makes it cacheable. The user message holds the growing transcript, the admissible actions, and a trailing prompt character. The example below uses the real initial observation of task 0.

System message (ALFWorld)

Interact with a household to solve a task. Output only the next line: either an action, or a line beginning with 'think:' to reason (a 'think:' line is not an environment action). Here are two examples.

<ReAct demonstration react_<type>_<i>>

```
<ReAct demonstration react_<type>_<j>>

Here is the task.

# Retrieved experience / skills
<33 insights in BM25 order, one per paragraph, 645 tokens in total>
```

User message (ALFWorld, first turn of task 0)

```
-- Welcome to TextWorld, ALFRED! --

You are in the middle of a room. Looking quickly around you, you see a bed 1, a desk 2, a desk 1, a drawer 6, a
  drawer 5, a drawer 4, a drawer 3, a drawer 2, a drawer 1, a garbagecan 1, a laundryhamper 1, a safe 1, a
  shelf 6, a shelf 5, a shelf 4, a shelf 3, a shelf 2, and a shelf 1.

Your task is to: put a mug in desk.
Valid actions right now: <admissible commands, separated by '; '>
>
```

The executor's reply is parsed as its first non-empty line with any leading > removed. A line beginning with think: is appended to the transcript and answered with OK. without consuming an environment step; any other line is executed. Because this ALFWorld version uses the verb move *X* to *Y*, a reply of the form put *X* in/on *Y* is rewritten to that grammar for every condition. Each subsequent user message is the full transcript (> action followed by the observation) plus the current admissible actions. An episode ends on success, after 30 environment steps, or after 60 executor calls.

WebShop agent (Thought/Action). WebShop uses the generic Thought/Action scaffold with two worked examples and the same memory header. The reply is parsed by its last Action: line.

System message (WebShop)

```
You are an agent solving an interactive text task using the ReAct framework.
At each step: first write one short line starting with 'Thought:' with your reasoning, then output exactly one
  line starting with 'Action:' containing a single admissible action. Output nothing after the Action line.

# Examples
Task: Find a red cotton shirt under $30.
Thought: I should search using the requested attributes.
Action: search[red cotton shirt]
Observation: Search results include product ids.
Thought: I should inspect the most relevant product.
Action: click[b012example]
Observation: Product page with options and Buy Now.
Thought: I should select the requested option before buying.
Action: click[red]
Observation: Red is selected.
Thought: The item matches, so I should purchase it.
Action: click[buy now]

<second example: a 32-ounce insulated water bottle, same format>

# Retrieved experience / skills
```

<WebShop rules in BM25 order, at most 1,024 tokens>

User message (WebShop)

<WebShop observation, including the instruction>

Valid actions right now
<admissible actions, separated by '; '>

Interaction so far
<previous Action/Observation pairs, or "(none yet)">

What is your next Thought and Action?

Memory distillation (building the pools). The pools are built offline from training-only trajectories with the following prompts. Compact insights use the first prompt once per task type with at most six successful trajectories; Markdown procedures use the second prompt once per successful trajectory; the WebShop pool uses the third prompt on train sessions 500–519.

Insight distillation (ExpeL-style, ALFWorld)

Below are successful agent trajectories for ALFWorld '<task_type>' tasks.
Extract 3 to 6 concise, GENERAL insights (rules/heuristics) that help solve this task type. Each insight: one imperative line, no specific object numbers or room layouts. Output ONLY a bullet list, each line starting with '- '.

Trajectories:
<up to six successful trajectories of this type, separated by '---'>

Procedure distillation (SkillOS-style Markdown, ALFWorld)

Below is a successful agent trajectory in ALFWorld.
Distill it into ONE reusable skill in Markdown with exactly:
'## Skill: <short name>', a 'When to use:' line, and a 'Steps:' numbered list (generalized – describe the procedure, not the specific object numbers). Keep it under 110 words.

Trajectory:
<one successful trajectory>

Rule distillation (WebShop)

You are distilling reusable procedural insights for the WebShop text environment.
The examples below come only from training sessions disjoint from evaluation.
Extract 6 to 10 concise general rules that improve product search, attribute checking, option selection, and purchase decisions. Do not mention product IDs, session IDs, or specific products. Output only bullet lines beginning with '- '.

Training experiences:
<positive-reward trajectories from train sessions 500–519>

Table 3. The 33-item experience memory used in every ALFWorld memory condition, in pool order. Counts per type: clean 5, cool 6, examine 6, heat 5, put 6, puttwo 5.

#	Type	Insight
0	clean	Identify the target item and its cleaning requirements before starting the task.
1	clean	Prioritize checking locations where the target item is likely to be found, such as countertops, cabinets, and drawers.
2	clean	Always clean the item using the designated cleaning area before placing it in the final location.
3	clean	Confirm the cleanliness of the item if necessary before moving it to the final destination.
4	clean	Ensure to follow the sequence of actions: find, clean, and then place the item correctly.
5	cool	Start by identifying likely locations for the target item based on its type.
6	cool	Always check the fridge first for items that need to be cooled.
7	cool	If the desired item is not found, systematically explore other potential locations.
8	cool	After obtaining the item, ensure to cool it before proceeding to the final placement.
9	cool	Confirm the availability of the final destination before moving the item there.
10	cool	Maintain a clear sequence of actions: find, cool, and place the item.
11	examine	Identify the required items and prioritize searching in likely locations based on their common placements.
12	examine	Check multiple potential locations systematically if the first search does not yield results.
13	examine	Use the most probable locations for the second required item after obtaining the first.
14	examine	Always confirm the presence of an item before attempting to use it.
15	examine	If an item is not found in one location, move to the next logical location without skipping options.
16	examine	Ensure to use the item in the correct context to achieve the task’s goal.
17	heat	Identify the target item and its required state before proceeding with the task.
18	heat	Prioritize checking locations where the target item is likely to be found.
19	heat	Use appropriate appliances or tools to modify the item as needed before completing the task.
20	heat	Ensure to open or interact with containers before placing or retrieving items.
21	heat	Maintain a clear sequence of actions to avoid unnecessary backtracking.
22	put	Start by checking locations where the target object is likely to be found, such as tables, countertops, or drawers.
23	put	If the initial location does not contain the target object, systematically explore other nearby locations.
24	put	Always check closed containers (drawers, cabinets) before assuming the object is not present.
25	put	Once the target object is found, plan the route to the designated location for placing it.
26	put	Ensure to confirm the target object is in your possession before attempting to place it.
27	put	When placing the object, verify that the destination is clear and appropriate for the item.
28	puttwo	Start by identifying the most likely locations for the target items based on their type.
29	puttwo	Check each location systematically, ensuring to open closed containers before searching inside.
30	puttwo	Once an item is found, immediately plan the next steps to retrieve and place it in the designated location.
31	puttwo	If the first search does not yield results, revisit previously checked locations, as items may be present in multiple instances.
32	puttwo	Keep track of items already found and placed to avoid unnecessary repetition in searches.

Decoding. All executor calls use temperature 0 and at most 256 completion tokens. Qwen3-8B is served by vLLM with `chat_template_kwargs = {"enable_thinking": false}`; GPT-4o-mini uses the provider defaults otherwise. Distillation calls use the same decoding with at most 512 completion tokens.

D EXPERIENCE MEMORY CONTENTS

Table 3 lists the complete focal experience memory, the 33 ExpeL-style insights distilled with the prompt above. Every item carries a task-type tag, and all 33 items (645 tokens) are injected on every task, so BM25 decides only their order. The order placebo keeps each item’s tag, index, and word count; the first [clean] item, for example, becomes [clean] the the item cleaning its starting requirements x and target Identify before (Appendix N).

Table 4. Hyperparameters held fixed across conditions.

Parameter	Value
Temperature	0
Completion cap per turn	256 tokens
Demonstrations per prompt	2 of 3 per task type
Step limit (ALFWorld / WebShop)	30 / 15
Memory budget B (ALFWorld / WebShop / placebo gate)	2,048 / 1,024 / 645
Item cap K	20
Qwen3-8B max model length	16,384
Bootstrap draws (ALFWorld / gate / WebShop)	10,000 / 20,000 / 20,000

E EXPERIMENTAL DETAILS

Benchmarks. ALFWorld (Shridhar et al., 2021) is evaluated on its 134-task unseen split, covering clean, cool, examine, heat, put, and put two, with binary success within 30 steps. WebShop (Yao et al., 2022) is evaluated in its small environment with 1,000 products (official commit 64fa2a5c15c7) and a 15-step limit; environment reward is the primary outcome and exact purchase success is secondary.

Baselines. The baseline for every stack is the no-memory harness W_0 . Exploratory memory conditions are raw successful trajectories and SkillOS-style Markdown procedures under the same insertion point and nominal budget; these are representation reproductions under one harness, not reimplementations of the source systems’ optimizers or curators. For content attribution, the baseline is the payload-matched order placebo of Section 2.6.

Models and protocol. Nominal seeds 0, 1, and 2 select demonstration pairs 02, 12, and 12; seed 5 was run to supply pair 01 (Appendix M). The focal contrast weights pairs 01, 02, and 12 equally and averages the two executions of pair 12. The placebo gate reuses the matching no-memory and true-memory episodes and runs only the placebo under both stacks with $B = 645$.

Hyperparameters and splits. Table 4 lists the settings held fixed across conditions. ALFWorld intervals resample the 134 task clusters 10,000 times; the placebo gate resamples ten tasks with replacement within each task type (20,000 draws, seed 20260713); WebShop uses 20,000 paired task-bootstrap draws, and its gate, split, primary outcome, and stopping rule were frozen before the 100-session comparison.

F PER-TASK-TYPE RESULTS AND COST

Tables 5 and 6 report the per-type success and the full token and cost accounting of the four runs behind the focal comparison in the nominal-seed matrix (402 episodes each: 134 tasks $\times$ seeds 0–2). The run summaries are released with the code.

With memory, GPT-4o-mini episodes are shorter (18.1 against 20.3 steps) and make 9.4% fewer calls, so although every call carries 645 more tokens, total prompt tokens rise by only 16.1% and dollar cost by 9.6% (\$2.36 against \$2.15 per 402 episodes). Because the memory sits in the stable system message, 85.0% of GPT-4o-mini prompt tokens are cached with memory against 79.1% without it. For Qwen3-8B the memory shortens episodes only slightly (10.3 against 11.1 steps) and raises prompt tokens by 25.4%.

Table 5. ALFWorld success by task type without memory, with the 645-token experience memory (compact insights + BM25), and the difference, for both executor stacks (nominal seeds 0–2). **Green:** gain; **red:** loss.

Task type	Episodes	GPT-4o-mini			Qwen3-8B		
		None	Memory	Δ	None	Memory	Δ
clean	93	0.452	0.667	+0.215	0.656	0.613	−0.043
cool	63	0.857	0.746	−0.111	0.857	0.857	0.000
examine	54	0.648	0.833	+0.185	0.611	0.574	−0.037
heat	69	0.130	0.377	+0.246	0.739	0.754	+0.014
put	72	0.833	0.917	+0.083	0.736	0.694	−0.042
puttwo	51	0.333	0.588	+0.255	0.431	0.431	0.000
All	402	0.540	0.687	+0.147	0.682	0.662	−0.020

Table 6. Token and cost accounting for the same four runs (402 episodes each). Token counts are in millions and summed over all calls. Qwen3-8B is served locally, so its dollar cost is zero and the runtime did not report cached tokens.

Executor	Memory	Calls	Steps/ep.	Prompt (M)	Cached (M)	Output (M)	Cost (\$)
 GPT-4o-mini	none	10,059	20.3	22.63	17.90	0.163	2.15
 GPT-4o-mini	645 tokens	9,115	18.1	26.26	22.31	0.151	2.36
 Qwen3-8B	none	12,380	11.1	32.43	–	0.506	0.00
 Qwen3-8B	645 tokens	12,262	10.3	40.68	–	0.491	0.00

Per type, the GPT-4o-mini gains are largest where its no-memory success is lowest (heat, puttwo, clean), and the only GPT-4o-mini loss is on its strongest type (cool); Qwen3-8B changes by at most 0.043 on any type.

G MODELS USED

Table 7 lists the models used in this paper. GPT-4o-mini and Qwen3-8B are the two executor stacks of the transfer study; GPT-4o-mini also distills the ALFWorld pools, Qwen3-8B distills the WebShop pool, and text-embedding-3-small is used only for dense ranking. “Access” distinguishes models served only behind an API from models whose weights are publicly released. The focal BM25 result does not depend on the embedding service.

H COMPARISON SCOPE

SKILLRC is an evaluation harness for experience memory, not a new memory method or harness optimizer, so it is not an empirical competitor to the systems discussed in Appendix I. Its comparisons are within-harness: every contrast changes one factor of Table 1 on identical tasks, so the paper makes no claim about the absolute performance of any released system. The executor contrast compares two *stacks*, and we do not claim that the difference is caused by model weights. The placebo is an experimental control with an oracle ranking reference, not a deployable retrieval scheme, and the representation matrix is asymmetric and exploratory rather than a ranking of representations.

I RELATED WORK

Table 7. Models used in this paper. Qwen3-8B is served with vLLM 0.24.0 from Hugging Face snapshot b968826d9c46 with enable_thinking=false; the GPT-4o-mini alias was queried in July 2026 and has no pinned snapshot.

Model	Provider	Access	Role
 GPT-4o-mini	OpenAI	Proprietary (API)	executor; ALFWorld distillation
 Qwen3-8B	Alibaba	Open weights	executor; WebShop distillation
 text-embedding-3-small	OpenAI	Proprietary (API)	dense ranking only

I.1 Experience-Driven Harness Optimization

Automatic improvement of LLM systems has targeted prompts (Fernando et al., 2023), declarative pipelines (Khattab et al., 2023), and agentic workflows (Zhang et al., 2024). Meta-Harness searches over harness code end to end (Lee et al., 2026), and MEMOHARNESS decomposes the harness into six editable dimensions, accumulates per-case entries and global patterns in a dual-layer experience bank, and adapts the learned harness to each test case (Huang et al., 2026). Agents that write experience into memory include Reflexion (Shinn et al., 2023) and ExpeL (Zhao et al., 2023), built on reasoning-and-acting scaffolds such as ReAct (Yao et al., 2023). These systems build or optimize the harness. SKILLRC instead isolates one of its components, the experience memory, and asks whether the gain that component produces survives a change of executor and a content control.

I.2 Skill Libraries, Curation, and Retrieval

SkillOpt treats a skill document as editable state accepted under held-out validation (Yang et al., 2026), SkillOS learns experience-driven skill curation with reinforcement learning around a frozen executor (Ouyang et al., 2026), and SelfEvoSkill revises skills from paired execution audits without external outcome supervision (Gao et al., 2026). SkillRet benchmarks retrieval from large public skill libraries (Kang et al., 2026), task-decomposition-guided reranking adapts skill retrieval to execution state (Chen et al., 2026), and experience reuse has been studied as continual learning that moves into memory (Hu et al., 2026). These systems build or select the memory; we study whether a fixed memory transfers and why.

I.3 Skill-Centered Evaluation

SkillsBench finds broad average gains from curated skills with negative deltas on some tasks (Li et al., 2026). A scalable framework evaluates agentic skills across many agent-model configurations (Shaposhnikov et al., 2026), and procedural-memory management has been studied across tasks, roles, and model backbones (Belikova et al., 2026). Realistic skill retrieval erodes gains (Liu et al., 2026), and SWE-Skills-Bench reports limited gains, mismatch harms, and token overhead in software engineering (Han et al., 2026). Related frameworks unify memory, skills, and rules as points on a compression spectrum (Zhang et al., 2026) and assess skills on utility, cost, and safety (Yu et al., 2026). Our narrower target is identification under a fixed artifact: a paired executor interaction combined with a payload-matched placebo.

I.4 Prompt Sensitivity and Distraction

Meaning-preserving formatting changes can swing accuracy substantially (Sclar et al., 2023), and irrelevant context degrades reasoning (Shi et al., 2023). The placebo result of RQ3 is an agentic instance

Table 8. Exploratory representation matrix over nominal seeds 0–2 against each stack’s matching no-memory harness, followed by executor contrasts (Qwen3-8B minus GPT-4o-mini). Intervals are task-clustered and unadjusted for multiplicity. **Green:** positive effect; **red:** negative effect.

Stack	Memory	Presentation	Paired Δ	95% CI	Tokens
GPT-4o-mini	compact insight	BM25	+0.1468	[+0.0846, +0.2090]	645
GPT-4o-mini	compact insight	dense	+0.0697	[+0.0174, +0.1244]	645
GPT-4o-mini	raw trajectory	BM25	+0.0721	[+0.0050, +0.1418]	1,779.7
GPT-4o-mini	raw trajectory	dense	+0.1045	[+0.0323, +0.1766]	1,779.0
GPT-4o-mini	Markdown	BM25	+0.0473	[−0.0025, +0.0995]	1,988.8
GPT-4o-mini	Markdown	dense	+0.0970	[+0.0398, +0.1542]	1,969.8
Qwen3-8B	compact insight	BM25	−0.0199	[−0.0721, +0.0299]	645
Qwen3-8B	compact insight	dense	−0.0050	[−0.0597, +0.0473]	645
Qwen3-8B	Markdown	BM25	−0.0771	[−0.1343, −0.0174]	1,988.8
Qwen3-8B	Markdown	dense	−0.0622	[−0.1244, −0.0025]	1,969.8
Executor $\times$ insight + BM25			−0.1667	[−0.2512, −0.0871]	
Executor $\times$ insight + dense			−0.0746	[−0.1567, +0.0025]	
Executor $\times$ Markdown + BM25			−0.1244	[−0.1990, −0.0498]	
Executor $\times$ Markdown + dense			−0.1592	[−0.2438, −0.0746]	
Executor $\times$ presentation (insight, triple difference)			−0.0920	[−0.1617, −0.0199]	

of this phenomenon: a lexically matched but incoherent block moves two executor stacks in opposite directions.

J REPRESENTATION MATRIX AND EXECUTOR INTERACTIONS

Table 8 reports the full exploratory matrix behind Table 2, with task-cluster intervals and the executor-by-memory difference-in-differences. Three of the four executor-by-memory contrasts exclude zero, and the presentation contrast for compact insights shows that order alone interacts with the executor. Integrity checks on the raw episode files confirmed 402/402 episodes per run, aligned task and seed keys, zero errors, and exact agreement between raw and stored success rates.

K TASK-LEVEL TRANSFER DIAGNOSTIC

A single transport gap tells us *whether* the memory transfers on average; it does not tell us *which* tasks carry the gap. Table 9 classifies each of the 134 ALFWorld tasks by the sign of its equal-pair memory effect under each stack and reports the mean effects and the mean Qwen3-8B no-memory success in each cell. Tasks that help GPT-4o-mini and hurt Qwen3-8B have the highest Qwen3-8B baseline (0.833), and tasks that help both have the lowest (0.407), the pattern summarized in RQ4. Representative cases include task 55 (put; GPT-4o-mini +0.833, Qwen3-8B +0.333) among tasks that help both, and task 133 (heat; GPT-4o-mini +0.833, Qwen3-8B −0.333, Qwen3-8B baseline 1.0) among tasks that help GPT-4o-mini and hurt Qwen3-8B. These labels prioritize cases for trajectory review; they do not identify mechanisms.

L CASE STUDIES

This appendix walks through what each kind of task asks the agent to do, what the experience memory offers for it, and what happened with and without the memory. Table 10 first summarizes the six

Table 9. Task-level transfer diagnostic over 134 ALFWorld tasks (compact insights + BM25, equal prompt pairs). Each row is a cell of Figure 4. **Green:** positive mean effect; **red:** negative mean effect.

GPT-4o-mini / Qwen3-8B	Tasks	GPT Δ	Qwen Δ	Qwen baseline
help / help	9	+0.444	+0.370	0.407
help / neutral	31	+0.452	0.000	0.699
help / hurt	14	+0.429	-0.357	0.833
neutral / help	12	0.000	+0.306	0.417
neutral / neutral	35	0.000	0.000	0.867
neutral / hurt	11	0.000	-0.364	0.636
hurt / help	5	-0.267	+0.333	0.333
hurt / neutral	11	-0.333	0.000	0.879
hurt / hurt	6	-0.333	-0.417	0.611

ALFWorld task families. Cases 1–3 then follow three individual tasks end to end; they are the only tasks whose goal, room, and per-episode outcomes were retained in the run logs. Cases 4–6 cover the remaining families at the level of the family and its representative tasks, and Case 7 covers WebShop.

Sources. Goals and room descriptions are the environment’s own text from our run logs. The *expert solution* is the walkthrough stored with each ALFWorld game file, read directly from the benchmark data without running any model; the expert knows where every object is, so its length is a lower bound for an agent that must search. Episode records come from GPT-4o-mini on seed 0 (demonstration pair 02). Task-level effects are the equal-pair effects of Table 9: success is averaged over demonstration pairs 01, 02, and 12, so -0.333 means that one of three pairs went from success to failure. “Rule rank” is the position, among the 33 injected items, of the first item tagged with the task’s own type.

What the figures show. ALFWorld gives the agent text only, but every ALFWorld game is generated from an ALFRED trial recorded in the AI2-THOR 3D simulator, and the 134 unseen tasks take place in just four rooms: a kitchen (77 tasks), a bedroom (27), a bathroom (19), and a living room (11). To show the environment behind the text, panel (a) of each ALFWorld figure renders the trial’s room with AI2-THOR 2.1.0, the version ALFWorld uses, after restoring the trial’s object positions and states. The left image is a first-person view chosen to show the places the expert solution visits; the right image is the room from above, with a cone marking where the left view was taken. Labels use the names that appear in the agent’s observation, such as shelf, desk, and garbagecan. Colored labels mark where the object starts (orange), the appliance applied to it (purple), and the destination (green), and the arrow on the map follows the expert’s route. Each goal exists as three trials in the same room that differ in where objects are placed. For Cases 1–3 we render a trial whose goal text matches the run log; for Cases 4–6, whose goals were not retained, we render an example task of the family drawn from the same 134 unseen tasks. Rendering runs no model. The WebShop figure shows screenshots of the official WebShop web application serving the benchmark’s 1,000-product catalogue. Its instruction is the one recorded for evaluation session 0; the search and clicks are ours, chosen to show a complete purchase, and the result ranking comes from a BM25 stand-in for the benchmark’s Lucene index.

Table 10. The six ALFWorld task families of the 134 unseen tasks. The expert procedure is the most common walkthrough pattern; expert steps are the mean and range of the benchmark walkthrough lengths. Success rates are from Table 5.

Family	Tasks	What the goal requires (expert procedure)	Expert steps	GPT-4o-mini	Qwen3-8B
put	24	find an object, take it, go to the target receptacle, place it	4.6 (4–6)	0.833 → 0.917	0.736 → 0.694
clean	31	find, take, go to the sink basin, <i>clean</i> , go to the target, place	6.3 (4–7)	0.452 → 0.667	0.656 → 0.613
heat	23	find, take, go to the microwave, <i>heat</i> , go to the target, place	6.0 (4–7)	0.130 → 0.377	0.739 → 0.754
cool	21	find, take, go to the fridge, <i>cool</i> , go to the target, place	6.1 (4–7)	0.857 → 0.746	0.857 → 0.857
examine	18	find an object, take it, go to a desk lamp, <i>use</i> the lamp	3.8 (3–4)	0.648 → 0.833	0.611 → 0.574
puttwo	17	find and place one instance, then find and place a second	8.6 (8–9)	0.333 → 0.588	0.431 → 0.431

Case 1 · put · Task 0: “put a mug in desk”

(a) The task and its room (AI2-THOR rendering of the benchmark trial)

Goal. Your task is to: put a mug in desk.

Initial observation. You are in the middle of a room. Looking quickly around you, you see a bed 1, a desk 2, a desk 1, a drawer 6, a drawer 5, a drawer 4, a drawer 3, a drawer 2, a drawer 1, a garbagecan 1, a laundryhamper 1, a safe 1, a shelf 6, a shelf 5, a shelf 4, a shelf 3, a shelf 2, and a shelf 1.



first-person view inside the room

the room from above; cone: the left view; arrow: the expert's route

Expert solution (4 steps): go to shelf → take mug from shelf → go to desk → move mug to desk

(b) What happened: GPT-4o-mini, seed 0

expert 4
no memory 19, solved
memory 6, solved

30-step limit

prompt tokens: 40,783 → 17,850
effect over 3 demo pairs:
GPT 0.000, Qwen −0.333

Figure 6. Task 0. (a) The bedroom of this task, rendered from one of the two benchmark trials whose goal text matches the run log. The mug starts on a shelf of the desk hutch and must be moved to the other desk; the expert needs four steps. (b) GPT-4o-mini on seed 0 solves the task in both conditions, in 19 steps without memory and 6 with it.

The task. The agent starts in a bedroom and reads: *“You are in the middle of a room. Looking quickly around you, you see a bed 1, a desk 2, a desk 1, a drawer 6, a drawer 5, a drawer 4, a drawer 3, a drawer 2, a drawer 1, a garbagecan 1, a laundryhamper 1, a safe 1, a shelf 6, a shelf 5, a shelf 4, a shelf 3, a shelf 2, and a shelf 1. Your task is to: put a mug in desk.”* The room has 18 places to look, and the agent does not know where the mug is.

What it takes. In the benchmark data this goal exists as three trials in the same room, and in all three the mug starts on a shelf. The expert solution is four steps: go to shelf, take mug from shelf, go to desk, move mug to desk. A searching agent needs extra steps to find the right shelf.

What the memory offers. Two [put] demonstrations are in the prompt in both conditions. With memory, the agent also reads all 33 insights; the first [put] rule appears third. The six [put] rules (items 22–27) describe a search-then-place routine, for example “If the initial location does not contain the target object, systematically explore other nearby locations” and “Ensure to confirm the target object is in your possession before attempting to place it”. Notably, the rule naming likely places lists “tables, countertops, or drawers”, not shelves.

What happened. GPT-4o-mini (seed 0) solves the task in both conditions: in 19 steps with 40,783 prompt tokens without memory, and in 6 steps, two above the expert, with 17,850 prompt tokens with memory. Over the three demonstration pairs, GPT-4o-mini’s success on this task is the same with and without memory (effect 0.000). Qwen3-8B solves it on every pair without memory and fails one pair with memory (−0.333).

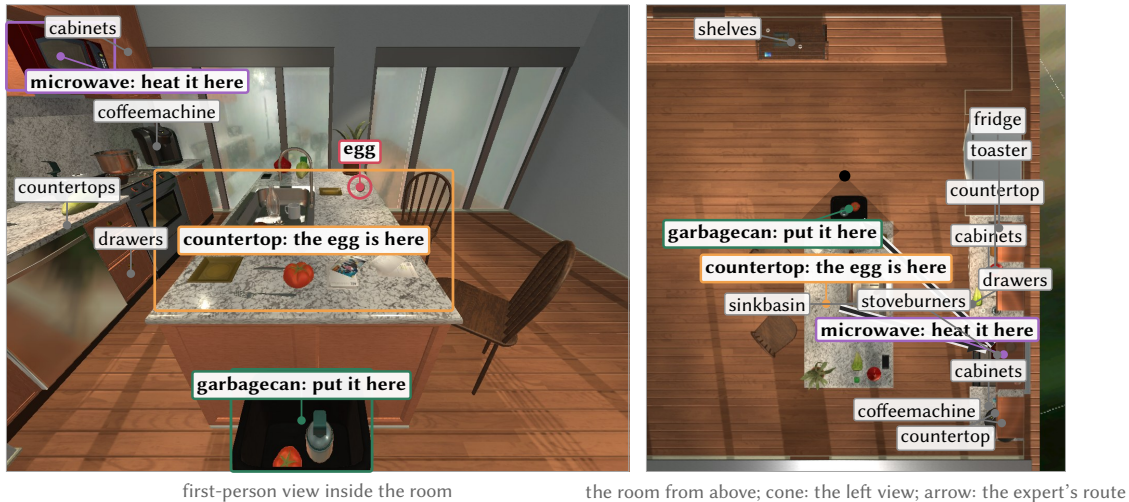
Analysis. The memory makes GPT-4o-mini’s successful search much more direct on this task, from more than four times the expert length to near it, but because the task was already solved, the success rate does not register the improvement. Without the step trace we cannot tell which rule, if any, shortened the search. For Qwen3-8B, which was at ceiling on this task, the added block could only cost success, and it did on one pair. Success-rate deltas therefore miss efficiency gains, and for a stack already at ceiling they can only record losses.

Case 2 · heat · Task 1: “heat some egg and put it in garbagecan”

(a) The task and its room (AI2-THOR rendering of the benchmark trial)

Goal. Your task is to: heat some egg and put it in garbagecan.

Initial observation. You are in the middle of a room. Looking quickly around you, you see a cabinet 6, a cabinet 5, a cabinet 4, a cabinet 3, a cabinet 2, a cabinet 1, a coffeemachine 1, a countertop 3, a countertop 2, a countertop 1, a drawer 3, a drawer 2, a drawer 1, a fridge 1, a garbagecan 1, a microwave 1, a shelf 3, a shelf 2, a shelf 1, a sinkbasin 1, a stoveburner 4, a stoveburner 3, a stoveburner 2, a stoveburner 1, and a toaster 1.



Expert solution (6 steps): go to countertop → take egg from countertop → go to microwave
heat egg with microwave → go to garbagecan → move egg to garbagecan

(b) What happened: GPT-4o-mini, seed 0

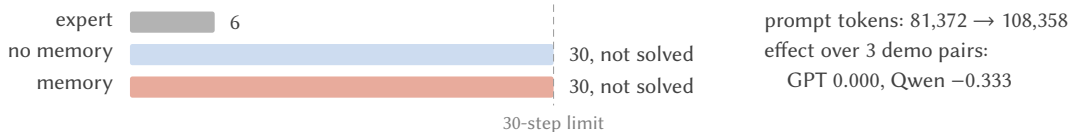


Figure 7. Task 1. (a) The kitchen, rendered from a trial in which the egg starts on a countertop (two of the three trials of this goal; in the third it starts in the garbage can). The egg must be heated in the microwave and then put in the garbage can. (b) GPT-4o-mini on seed 0 reaches the 30-step limit in both conditions.

The task. The agent starts in a kitchen: “... you see a cabinet 6, a cabinet 5, a cabinet 4, a cabinet 3, a cabinet 2, a cabinet 1, a coffeemachine 1, a countertop 3, a countertop 2, a countertop 1, a drawer 3, a drawer 2, a drawer 1, a fridge 1, a garbagecan 1, a microwave 1, a shelf 3, a shelf 2, a shelf 1, a sinkbasin 1, a stoveburner 4, a stoveburner 3, a stoveburner 2, a stoveburner 1, and a toaster 1. Your task is to: heat some egg and put it in garbagecan.” There are 25 places to look.

What it takes. The egg starts on a countertop in two of the three trials of this goal and inside the garbage can in the third. The expert solution is six steps: go to the egg, take it, go to microwave, heat egg with microwave, go to garbagecan, move egg to garbagecan.

What the memory offers. Here the first [heat] rule is the *first* item of the memory block, so the matching procedure is as salient as it can be. The five [heat] rules (items 17–21) include “Use appropriate appliances or tools to modify the item as needed before completing the task” and “Ensure to open or interact with containers before placing or retrieving items”.

What happened. GPT-4o-mini (seed 0) fails in both conditions, stopping at the 30-step limit, five

times the expert length. The memory raises the episode’s prompt tokens from 81,372 to 108,358. Over the three pairs, GPT-4o-mini’s success on this task is unchanged (0.000), while Qwen3-8B, which solved it on one pair without memory, loses that pair (−0.333).

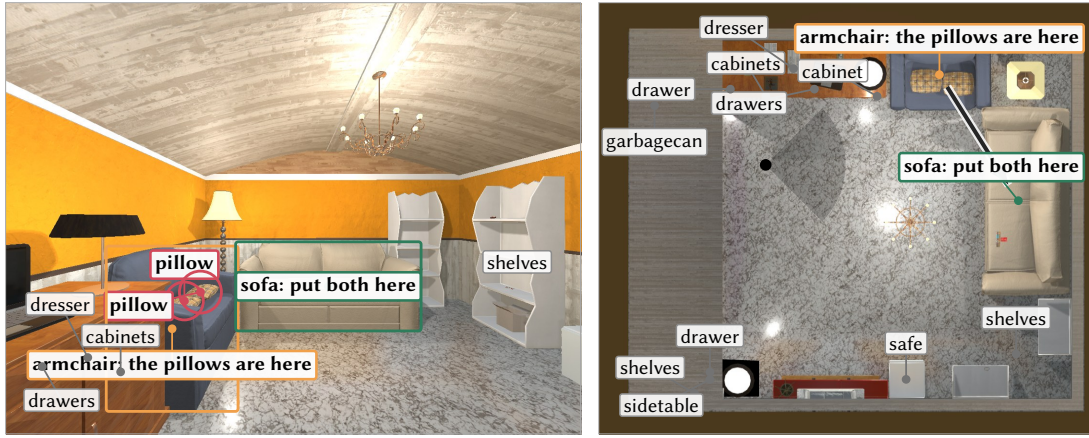
Analysis. heat is GPT-4o-mini’s weakest family (0.130 without memory) and the one where the memory helps it most after puttwo (+0.246), so this task is a counterexample inside a strong family gain: the most relevant rule comes first and the agent still fails, at a 33% higher token cost. The rules state the generic procedure (find, use the appliance, place) but nothing about where an egg is, which is what the agent must discover. Task 133, another heat task with its first rule also at rank 1, shows the family’s other face: GPT-4o-mini +0.833, Qwen3-8B −0.333 from a Qwen3-8B no-memory success of 1.000.

Case 3 · puttwo · Task 2: “put two pillow in sofa”

(a) The task and its room (AI2-THOR rendering of the benchmark trial)

Goal. Your task is to: put two pillow in sofa.

Initial observation. You are in the middle of a room. Looking quickly around you, you see a armchair 1, a cabinet 4, a cabinet 3, a cabinet 2, a cabinet 1, a drawer 5, a drawer 4, a drawer 3, a drawer 2, a drawer 1, a dresser 1, a garbagecan 1, a safe 1, a shelf 12, a shelf 11, a shelf 10, a shelf 9, a shelf 8, a shelf 7, a shelf 6, a shelf 5, a shelf 4, a shelf 3, a shelf 2, a shelf 1, a sidetable 1, and a sofa 1.



first-person view inside the room

the room from above; cone: the left view; arrow: the expert’s route

Expert solution (8 steps): go to armchair → take pillow from armchair → go to sofa → move pillow to sofa → go to armchair → take pillow from armchair → go to sofa → move pillow to sofa

(b) What happened: GPT-4o-mini, seed 0

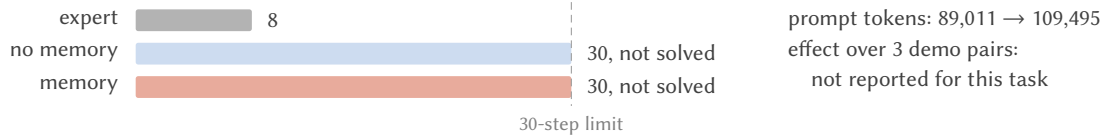


Figure 8. Task 2. (a) The living room, rendered from one of the two trials whose goal text matches the run log. Both pillows start on the armchair and must be moved to the sofa, so the expert repeats one four-step routine. (b) GPT-4o-mini on seed 0 reaches the 30-step limit in both conditions.

The task. The agent starts in a living room with 27 places to look: “... you see a armchair 1, a cabinet 4, a cabinet 3, a cabinet 2, a cabinet 1, a drawer 5, a drawer 4, a drawer 3, a drawer 2, a drawer

1, a dresser 1, a garbagecan 1, a safe 1, a shelf 12, ..., a shelf 1, a sidetable 1, and a sofa 1. Your task is to: put two pillow in sofa."

What it takes. In all three trials of this goal both pillows start on the armchair. The expert solution is eight steps, the same four-step routine performed twice: go to armchair, take pillow from armchair, go to sofa, move pillow to sofa.

What the memory offers. The five [puttwo] rules (items 28–32) speak directly to the difficulty of the family, which is keeping track of the second instance: "Once an item is found, immediately plan the next steps to retrieve and place it", "If the first search does not yield results, revisit previously checked locations, as items may be present in multiple instances", and "Keep track of items already found and placed to avoid unnecessary repetition in searches". The first [puttwo] rule is ranked 4th for the family's representative tasks.

What happened. GPT-4o-mini (seed 0) fails in both conditions at the 30-step limit, nearly four times the expert length, and the memory raises prompt tokens from 89,011 to 109,495. This task is not among the representative cases of Table 9, so its equal-pair effects are not reported.

Analysis. Even with rules that name the exact difficulty and an expert path that needs only one location, GPT-4o-mini does not finish this task within the budget on seed 0. Across the family, however, the memory lifts GPT-4o-mini more than any other family ($0.333 \rightarrow 0.588$) while leaving Qwen3-8B exactly unchanged ($0.431 \rightarrow 0.431$); two representative tasks move in opposite directions for the two stacks (task 103: GPT-4o-mini +1.000, Qwen3-8B 0.000; task 118: GPT-4o-mini -0.333, Qwen3-8B +0.333). The transport gap is therefore visible inside a single family.

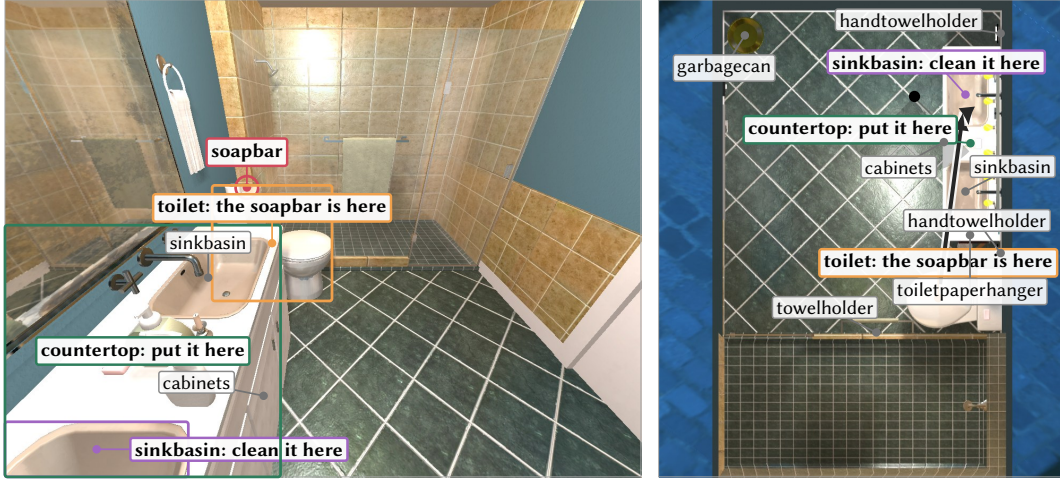
The goals of the remaining representative tasks were not retained in the run logs, so Cases 4–6 describe each family through the benchmark data and identify its representative tasks by ID. Their figures show an example task of the family instead.

Case 4 · clean · family of 31 tasks; representative task 30

(a) An example task of this family and its room (AI2-THOR rendering)

Goal. Your task is to: put a clean soapbar in countertop.

Initial observation. You are in the middle of a room. Looking quickly around you, you see a cabinet 4, a cabinet 3, a cabinet 2, a cabinet 1, a countertop 1, a garbagecan 1, a handtowelholder 2, a handtowelholder 1, a sinkbasin 2, a sinkbasin 1, a toilet 1, a toiletpaperhanger 1, and a towelholder 1.



Expert solution (6 steps): go to toilet → take soapbar from toilet → go to sink
clean soapbar with sink → go to countertop → move soapbar to countertop

(b) Effect of the memory on this family, 3 demonstration pairs

family success: GPT 0.452 → 0.667, Qwen 0.656 → 0.613

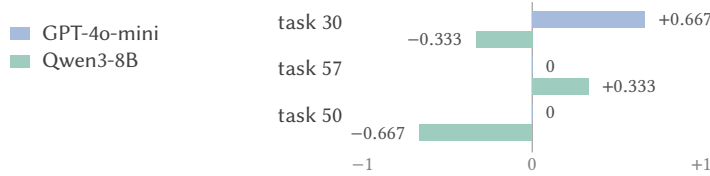


Figure 9. The clean family. (a) An example clean task from the benchmark, in the bathroom: the soap bar starts on the toilet, must be cleaned at the sink basin, and goes on the countertop. It is not task 30, whose goal was not retained. (b) Memory effect on three representative tasks for each executor, averaged over the three demonstration pairs.

The tasks. The agent must clean an object and place it somewhere, in kitchens and bathrooms. The most frequent objects in the 31 unseen clean tasks are soap bars, cloths, plates, eggs, and mugs; the most frequent destinations are countertops and cabinets.

What it takes. Six steps on average (4–7): find the object, take it, go to sinkbasin, clean <object> with sinkbasin, go to the destination, and place it.

What the memory offers. The five [clean] rules (items 0–4), including “Always clean the item using the designated cleaning area before placing it in the final location” and “Ensure to follow the sequence of actions: find, clean, and then place the item correctly”. The first [clean] rule is ranked 4th for task 30.

What happened. Family: GPT-4o-mini 0.452 → 0.667, Qwen3-8B 0.656 → 0.613. Task 30: GPT-4o-mini +0.667 (two of three pairs gained), Qwen3-8B -0.333 from a no-memory success of

0.333 (its only solved pair is lost). Two other clean tasks: task 57 (GPT-4o-mini 0.000, Qwen3-8B +0.333) and task 50 (GPT-4o-mini 0.000, Qwen3-8B -0.667).

Analysis. The rules spell out the very step this family adds, and GPT-4o-mini, which starts below 50%, gains 0.215. Task 30 shows the transport gap on a task where Qwen3-8B is *not* at ceiling, so headroom alone does not explain it, and across three clean tasks the same block helps Qwen3-8B once and hurts it twice. Under the order placebo, the same five [clean] items appear in the same positions as word salad (Table 11), and the placebo alone already raises GPT-4o-mini and lowers Qwen3-8B on average (RQ3).

Case 5 · cool · family of 21 tasks; representative task 5

(a) An example task of this family and its room (AI2-THOR rendering)

Goal. Your task is to: put a cool mug in cabinet.

Initial observation. You are in the middle of a room. Looking quickly around you, you see a cabinet 6, a cabinet 5, a cabinet 4, a cabinet 3, a cabinet 2, a cabinet 1, a coffeemachine 1, a countertop 3, a countertop 2, a countertop 1, a drawer 3, a drawer 2, a drawer 1, a fridge 1, a garbagecan 1, a microwave 1, a shelf 3, a shelf 2, a shelf 1, a sinkbasin 1, a stoveburner 4, a stoveburner 3, a stoveburner 2, a stoveburner 1, and a toaster 1.



Expert solution (6 steps): go to countertop → take mug from countertop → go to fridge → cool mug with fridge → go to cabinet → move mug to cabinet

(b) Effect of the memory on this family, 3 demonstration pairs

family success: GPT 0.857 → 0.746, Qwen 0.857 → 0.857

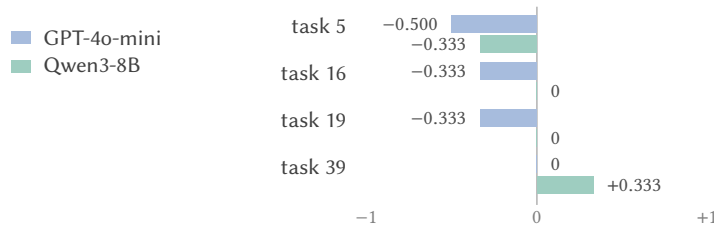


Figure 10. The cool family. (a) An example cool task, in the kitchen: the mug starts on a countertop, must be cooled in the fridge, and goes into a cabinet. (b) Memory effect on four representative tasks.

The tasks. The agent must cool an object in the fridge and place it somewhere, typically in a kitchen. Frequent objects are mugs, lettuce, pans, bread, and potatoes; frequent destinations are

countertops and the microwave.

What it takes. Six steps on average (4–7): find the object, take it, go to fridge, cool <object> with fridge, go to the destination, and place it.

What the memory offers. Six [cool] rules (items 5–10), such as “Always check the fridge first for items that need to be cooled” and “Maintain a clear sequence of actions: find, cool, and place the item”. The first [cool] rule is ranked 2nd for task 5.

What happened. Family: GPT-4o-mini $0.857 \rightarrow 0.746$, the only family in which GPT-4o-mini loses; Qwen3-8B $0.857 \rightarrow 0.857$. Task 5: GPT-4o-mini -0.500 and Qwen3-8B -0.333 , with Qwen3-8B solving it on every pair without memory; it is one of the six tasks harmed under both stacks. Other cool tasks: tasks 16 and 19 (GPT-4o-mini -0.333 , Qwen3-8B 0.000) and task 39 (GPT-4o-mini 0.000 , Qwen3-8B $+0.333$).

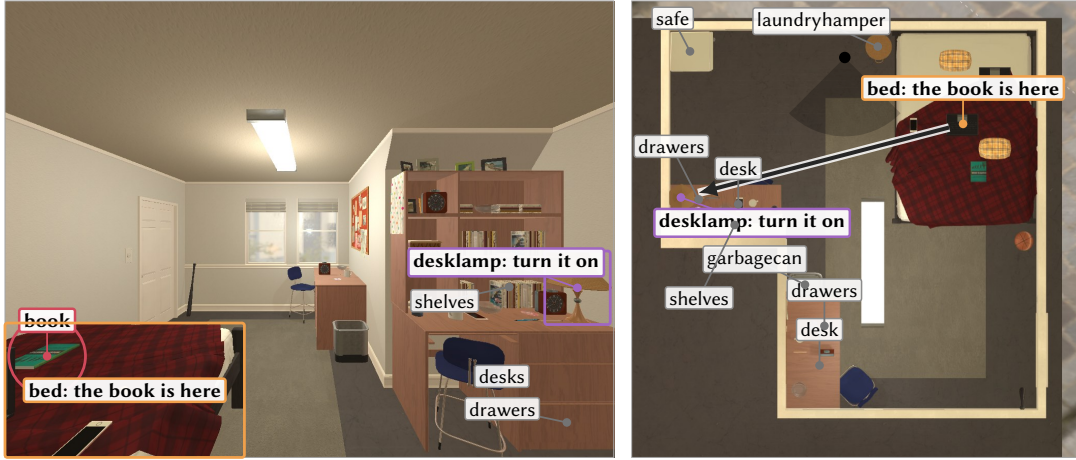
Analysis. cool is GPT-4o-mini’s strongest family without memory, and it is where the memory hurts GPT-4o-mini. The rules restate a procedure both executors already follow, and one of them (“Always check the fridge first”) points the search at the fridge, where the object must be cooled but is not necessarily found; the retained data cannot show whether this redirected the search. Harm concentrates where the no-memory harness is already strong, for either executor.

Case 6 · examine · family of 18 tasks; representative task 7

(a) An example task of this family and its room (AI2-THOR rendering)

Goal. Your task is to: examine the book with the desk lamp.

Initial observation. You are in the middle of a room. Looking quickly around you, you see a bed 1, a desk 2, a desk 1, a drawer 6, a drawer 5, a drawer 4, a drawer 3, a drawer 2, a drawer 1, a garbagecan 1, a laundryhamper 1, a safe 1, a shelf 6, a shelf 5, a shelf 4, a shelf 3, a shelf 2, and a shelf 1.



first-person view inside the room

the room from above; cone: the left view; arrow: the expert's route

Expert solution (4 steps): go to bed → take book from bed → go to desk → use desk lamp

(b) Effect of the memory on this family, 3 demonstration pairs

family success: GPT 0.648 → 0.833, Qwen 0.611 → 0.574

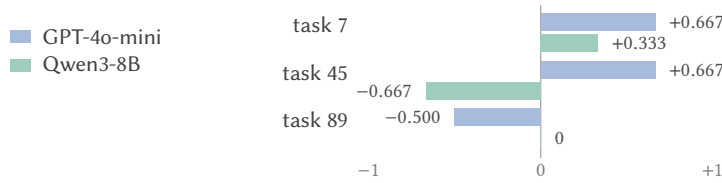


Figure 11. The examine family. (a) An example examine task, in the bedroom: the book lies on the bed, and the agent must carry it to the desk and turn on the desk lamp while holding it. (b) Memory effect on three representative tasks.

The tasks. The agent must look at an object under a desk lamp, in bedrooms. Frequent objects are mugs, pencils, bowls, CDs, and books.

What it takes. The shortest family, 3.8 steps on average (3–4): find the object, take it, go to the desk lamp, and use desk lamp. The difficulty is finding the object and recognizing that “examine” means turning on the lamp while holding it.

What the memory offers. Six [examine] rules (items 11–16), including “Use the most probable locations for the second required item after obtaining the first” and “Ensure to use the item in the correct context to achieve the task’s goal”. For task 7 the first [examine] rule is ranked only 7th.

What happened. Family: GPT-4o-mini 0.648 → 0.833, Qwen3-8B 0.611 → 0.574. Task 7: GPT-4o-mini +0.667 and Qwen3-8B +0.333 from a Qwen3-8B no-memory success of 0.333, one of only nine tasks that improve under both stacks. Task 45, of the same family, shows the reversal instead (GPT-4o-mini +0.667, Qwen3-8B -0.667, from a Qwen3-8B no-memory success of 0.667), and task 89 harms GPT-4o-mini (-0.500).

Analysis. Joint improvement happens on a task where both executors start low, even though the matching rule is far down the block. With a higher Qwen3-8B baseline, the same family produces the reversal. Baseline competence, not rule position, best separates the two outcomes in these cases.

Case 7 · WebShop · 100 train-disjoint sessions

(a) One WebShop session in the official web app (evaluation session 0)

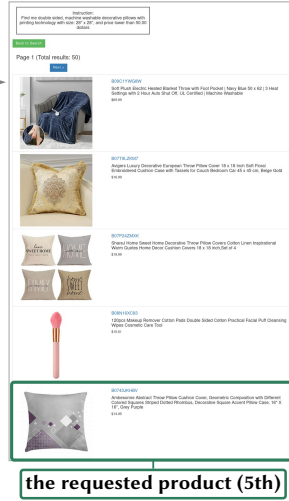
The start page shows the request and a search box.

What the reward checks. The purchase is compared with the request:

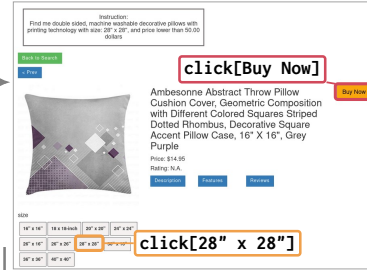
- attributes: double sided, machine washable, printing technology
- option: size 28" x 28"
- price: lower than \$50.00
- product type: the title must match the request

The reward is the share of matched attributes, options, and price, times the type match; here everything matches, so it is 1.0. A session allows at most 15 actions, and the store holds 1,000 products. The ranking shown comes from our BM25 stand-in for the search index.

After search[double sided machine washable decorative pillows 28 x 28]: 50 results, 10 per page.



After click[B0743JKHBV]: the product page, with its size options and a Buy Now button.



After click[28" x 28"] and click[Buy Now]: the episode ends; the score box (green) shows reward 1.0.

(b) What happened: Qwen3-8B, 100 sessions

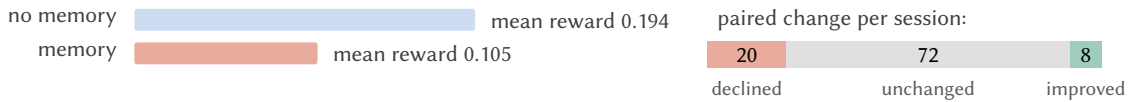


Figure 12. WebShop. (a) One session in the official web app: the start page with the request, the results of a search, the product page where the size is chosen, and the score page after the purchase. The query and clicks are ours; the reward of 1.0 shows what a full match looks like. (b) Mean reward and the sign of the paired reward change over 100 sessions.

The tasks. Each WebShop session gives a natural-language request for a product with required attributes, options, and a price limit, in a simulated store of 1,000 products. The agent searches, opens product pages, selects options, and buys; the environment reward measures how well the purchased product matches the request, and exact success requires a full match. Episodes are limited to 15 steps.

What the memory offers. Ten rules about search, attribute checking, option selection, and purchase, distilled by Qwen3-8B with the prompt of Appendix C from the six positive-reward trajectories of train sessions 500–519, and presented by BM25 with $B = 1,024$.

What happened. A 20-session gate confirmed valid execution (100% valid action syntax, search on 20/20, 13 purchase attempts, mean reward 0.288). On sessions 0–99, reward falls from 0.194 to

Table 11. Examples of the order placebo. Task tags, item indices, word counts, retrieval order, and total footprint are preserved.

True insight	Index-aligned order placebo
[clean] Identify the target item and its cleaning requirements before starting the task.	[clean] the the item cleaning its starting requirements x and target Identify before
[clean] Always clean the item using the designated cleaning area before placing it in the final location.	[clean] item final area it the Always the before designated clean in x using placing cleaning the

0.105 (paired -0.090 , $[-0.165, -0.016]$) and exact success from 0.09 to 0.07; 8 sessions improve, 20 decline, and 72 are unchanged.

Analysis. The experience behind the memory is thin, six successful trajectories, and the executor reading it is the one that showed no ALFWorld gain. The rules lower reward on twice as many sessions as they raise it. This case has one executor and no placebo, so it shows negative transfer but cannot separate content from context.

M SEED-AXIS AUDIT

The scaffold selects two of three demonstrations per task type with a seeded shuffle. Nominal seeds 0, 1, and 2 map to pairs 02, 12, and 12, so seeds 1 and 2 repeat one prompt condition and pair 01 was missing from the original matrix. Seeds 1 and 2 disagree on 20.9% of GPT-4o-mini no-memory task outcomes but on only 1.5% of Qwen3-8B outcomes despite identical prompts, which reflects residual hosted nondeterminism. Weighting the two observed pairs equally gives a GPT-4o-mini gain of $+0.134$ ($[+0.069, +0.200]$), a Qwen3-8B change of -0.035 ($[-0.082, +0.011]$), and a gap of -0.170 ($[-0.252, -0.088]$). After seed 5 supplied pair 01, the complete three-pair estimates are those reported in RQ1 and RQ2. Repeated executions of one prompt are averaged within pair and never treated as independent seeds.

N ORDER-PLACEBO CONSTRUCTION

The placebo is built in three deterministic steps: (i) shuffle the body words of each item, re-drawing until no original adjacent body bigram remains; (ii) greedily replace words with the nonce x until the payload measures exactly 645 tokens under both the GPT-4o-mini and Qwen3-8B tokenizers (28 of 472 words); (iii) serve the payload through the ranking-reference path so that BM25 scores are computed on the original text. The released builder regenerates the placebo exactly (645 tokens before and 673 after shuffling, 28 substitutions, 94.07% of body words retained, 645/645 tokens under both tokenizers), and the runtime audit confirmed identical order on 134/134 queries and all 33 items present. Table 11 shows two items.